



AI-Powered Content Moderation Pipeline

Scale trust & safety without scaling the moderation team linearly.

Multi-category classification

Context-aware analysis

Confidence-based routing

Explainable decisions

Human review queue

Per-platform policy

Assignment 10 · Trust & Safety | Stack: TypeScript / Node (Express) + React (Vite) | AI: Claude API (claude-opus-4-8)

1. What is this project?

This is a **multi-stage content-moderation pipeline** that decides, automatically and at scale, whether a piece of user-generated content is safe to publish, should be blocked, or needs a human to look at it.

Content moderation at scale is an unsolved operational problem. Simple keyword filters and naive classifiers produce far too many false positives, can't tell the difference between a genuine threat and friendly banter, and give no auditable reason for their decisions. This system addresses all three: it uses a large language model (Claude) for nuanced, context-aware classification, a *tunable* policy engine so every platform can set its own bar, and an *explainable* decision record that users and legal teams can audit.

2. What is it used for? (Use cases)

- **Social platforms & forums** — automatically screen posts, comments, and messages for harmful content before they reach other users.
- **Reducing moderator workload** — clear violations are actioned automatically; only genuinely ambiguous cases reach a human, so the team scales sub-linearly with content volume.
- **Multi-tenant / white-label products** — one engine serves many platforms, each with its own policy (a children's app and an adult community need very different rules).
- **Auditable / compliant moderation** — every decision stores the exact offending text, the harm category, and the reasoning, so it can be defended to users, regulators, or legal teams.
- **A feedback loop for model improvement** — human reviewers' final calls are captured as labelled data to improve the classifier over time.

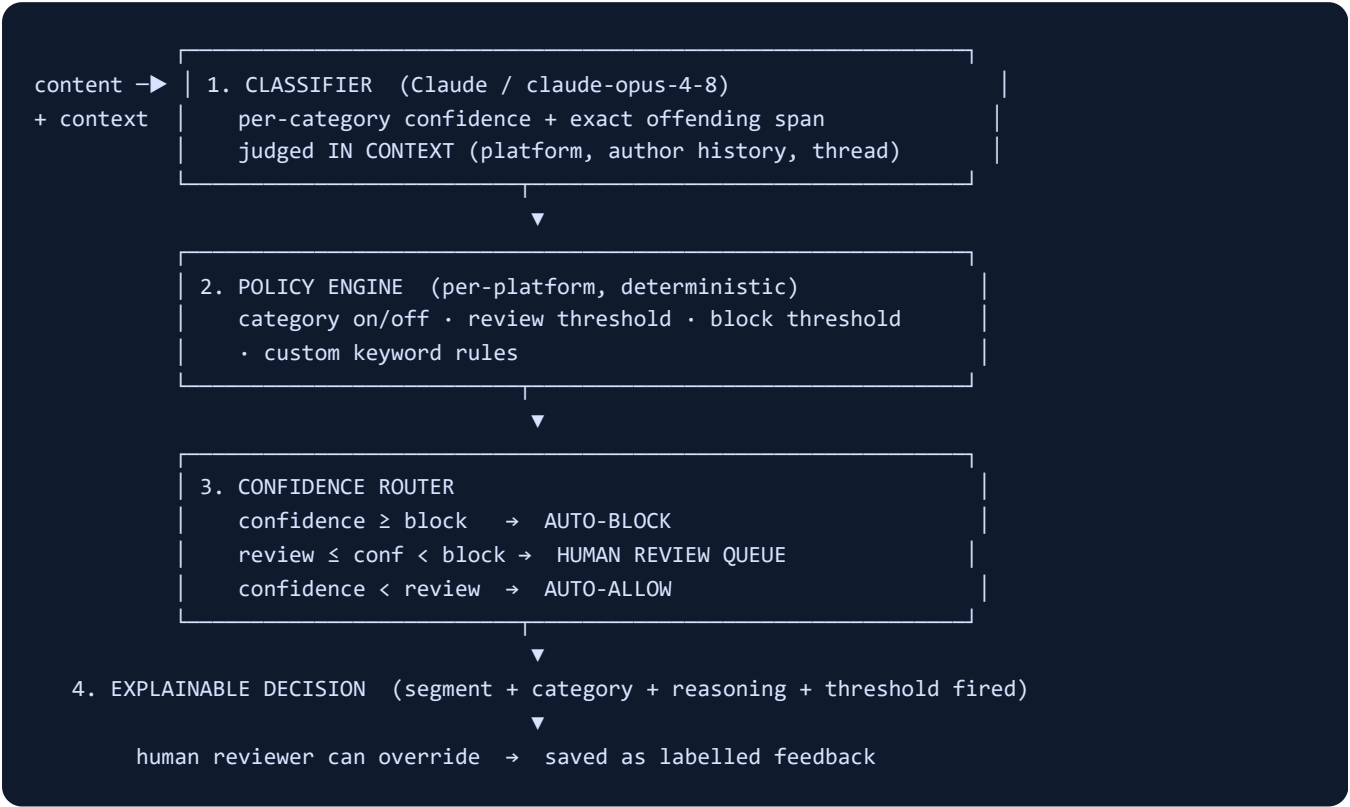
3. The seven harm categories

Every piece of content is scored, with a confidence value, against each of these categories:

Category	What it catches
Hate speech	Attacks or dehumanisation based on a protected attribute (race, religion, gender, etc.).
Harassment	Targeted bullying, threats, intimidation, or doxxing aimed at an individual.
Spam	Scams, mass advertising, link-farming, unsolicited promotion.
Misinformation	Demonstrably false or dangerous claims (health, elections, safety).
Graphic violence	Vivid depictions or glorification of gore or physical harm.
Adult content	Sexually explicit material or solicitation.
Self-harm	Content encouraging, instructing, or expressing intent toward suicide or self-injury.

4. How it works

Content flows through four stages. Crucially, **only the first stage calls the AI model** — everything after it is pure, deterministic code, which makes the system fast, cheap to re-run, predictable, and easy to test.



Stage 1 — Context-aware classification

The content, plus its *context* (which platform, the author's history, the conversation thread), is sent to Claude. The model returns a structured result: for each detected harm category a confidence score (0–1), a severity, the **verbatim offending segment**, and a one-line reason. Context matters enormously — "I'm going to kill you" between friends after a game is banter, but the same words sent to a stranger on a children's platform is a credible threat. The model is told to weigh exactly that.

Stage 2 — Policy engine (the tunable part)

Each platform has its own policy: every category can be turned on/off and has two thresholds — a *review* threshold and an *auto-block* threshold — plus optional custom keyword rules. The same classification result produces different outcomes under different policies.

Stage 3 — Confidence-based routing

Model confidence vs. policy	Routing
≥ auto-block threshold	AUTO-BLOCK — actioned automatically
between review and block	HUMAN REVIEW — sent to the queue

< review threshold

AUTO-ALLOW — published

Stage 4 — Explainable decision & human review

Every decision is stored with the offending segment highlighted, the category, the model's reasoning, and exactly which threshold or rule fired. Items routed to review appear in a moderator queue with the full context and the AI's recommendation; the reviewer's final call is recorded — and any disagreement with the AI is captured as labelled feedback for improving the model.

5. Example: same words, different context & policy

Context decides	Policy decides
Statement: "I'm going to kill you, you're dead next round."	Statement: an explicit adult short story.
- Gaming voice chat between friends → ALLOW - DM to a 12-year-old after an argument, sender has prior strikes → BLOCK / REVIEW	- Adult / Mature community (adult content disabled) → ALLOW - Children's platform (strict thresholds) → BLOCK

6. Technology used

Layer	Technology	Why
Language	TypeScript	Type-safe across backend and frontend; one shared domain model.
AI model	Claude API — claude-opus-4-8	Nuanced, context-aware classification with adaptive thinking.
AI integration	@anthropic-ai/sdk + structured outputs (Zod)	Forces the model to return schema-valid JSON — no brittle parsing.
Backend	Node.js + Express	Lightweight REST API hosting the pipeline.
Validation	Zod	Validates API requests and the model's structured output.
Frontend	React + Vite	Fast, interactive dashboard for moderation, the review queue, and policy config.
Storage	In-memory store	Dependency-free and resettable for the demo (swappable for SQLite/Postgres).

7. What was built (components)

- Classifier** — Claude call with a context-aware prompt and structured output.
- Policy engine** — applies per-platform thresholds + custom rules, produces explainable per-flag decisions.
- Pipeline + router** — orchestrates classify → policy → route → store.
- REST API** — endpoints for moderation, the review queue, reviews/feedback, and policy management.
- React dashboard** — three screens: *Moderate* (submit + see explained decision), *Review queue* (moderator workspace with overrides), *Policies* (sliders, toggles, custom rules).

- **Evaluation harness** — a labelled test set + runner that checks every success metric (classification coverage, context-sensitivity, routing, explainability, policy sensitivity).

8. How the success metrics are met

Requirement	How it's satisfied
Identifies all harm categories	Seven-category classifier; labelled test set covers each + clean controls.
Same statement, two contexts	Identical text in banter vs. threat context yields different actions.
Correct routing	Clear violations auto-block, clean content auto-allows, ambiguous goes to the queue.
Explainability	Every flag cites the verbatim offending segment + category + reasoning.
Policy changes behaviour	Identical content allowed on the adult platform, blocked on the children's platform.

9. How to run it

Backend: `cd backend` → `npm install` → copy `.env.example` to `.env` and add your `ANTHROPIC_API_KEY` → `npm run dev` (port 4000).

Frontend: `cd frontend` → `npm install` → `npm run dev` → open `http://localhost:5173` .

Prove the metrics: `cd backend` → `npm run evaluate` (runs the labelled set through the real pipeline and reports pass/fail).